RL Foundations: MDPs & Bellman Equations

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 14, 2026

Reinforcement Learning

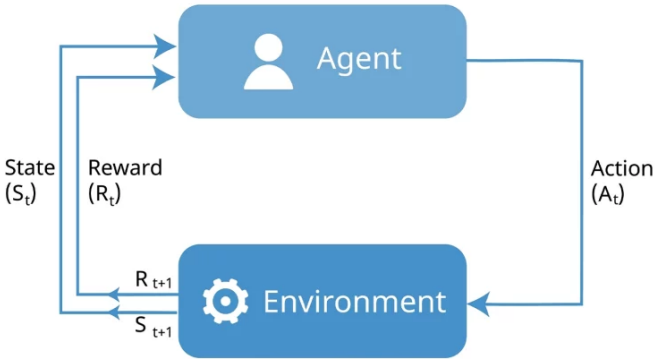
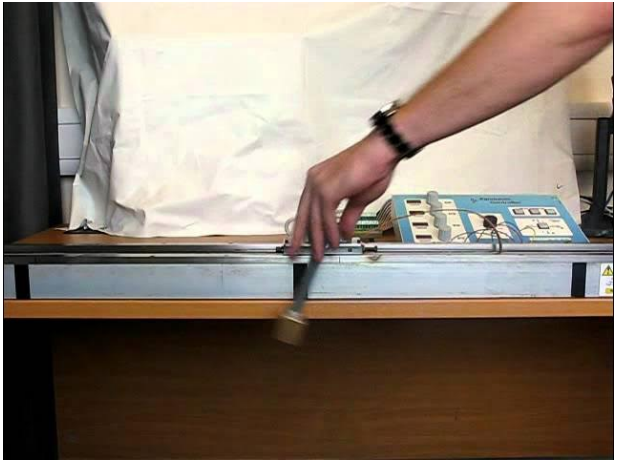


Table of Contents

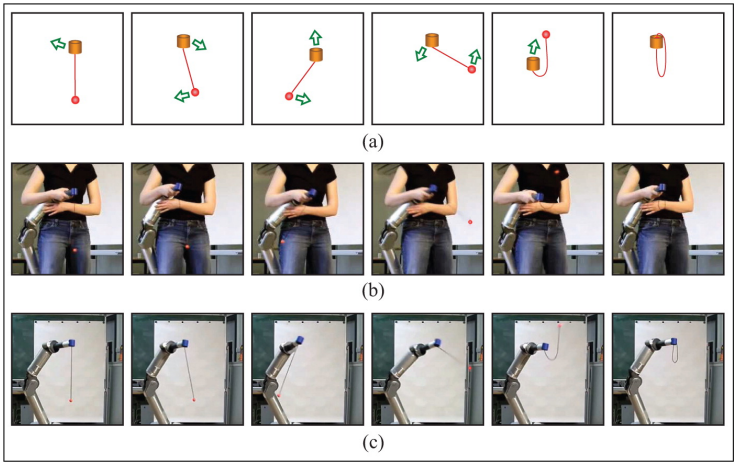
Table of Contents (cont.)

Motivation



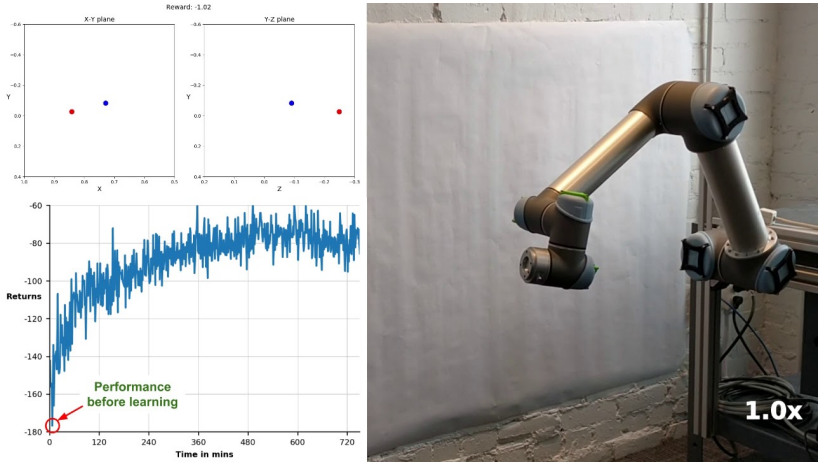
Cart-Pole Swing-Up Task

Motivation (cont.)



Robot Ball-in-a-Cup Task (Learning Motor Primitives in Robotics)

Motivation (cont.)

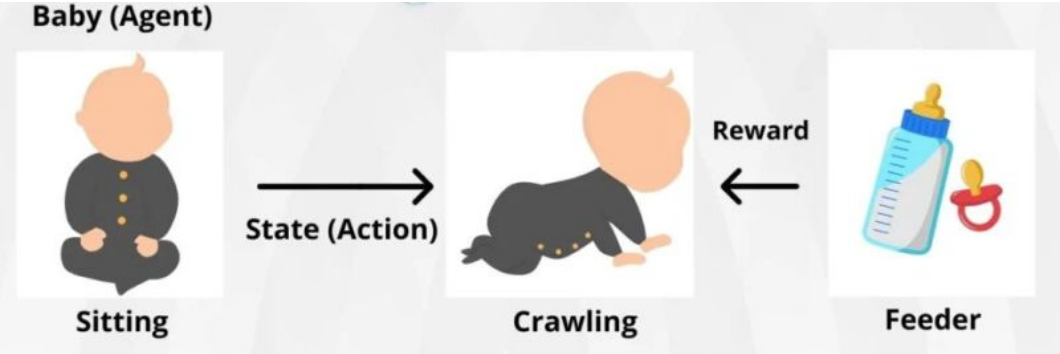


Robot Learning in Real-World Environments

Motivation (cont.)

- Many AI problems involve **sequential decision-making** under uncertainty.
- Inspired by **animal learning** (reward-based adaptation).
- Reinforcement Learning (RL) allows agents to learn optimal behaviors by **trial and error**.
- Real-world success: AlphaGo, Robotics, Game AI, Recommendation Systems.
- RL was the most-appearing keyword at ICLR 2023, reflecting how central it has become to modern AI research.

Motivation (cont.)



Baby Learning to Crawl towards a Milk Bottle (Reward)

Learning Outcomes

By the end of this session, you will be able to:

- Understand the fundamental **principles and structure of RL**.
- Model problems using **Markov Decision Processes (MDPs)**.
- Implement and apply **Q-learning** to solve control problems.
- Identify **practical applications and challenges** in RL.
- Critically analyze the **limitations and future trends** of RL.

Reinforcement Learning: **Introduction**

Definition: RL is a type of machine learning where agents learn to make decisions by interacting with an environment.

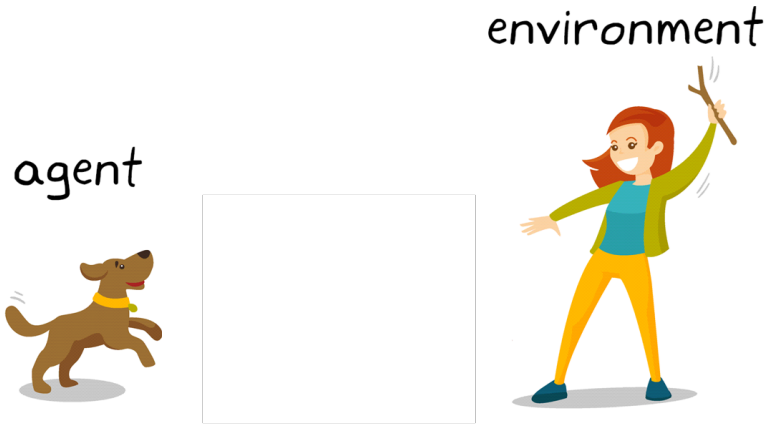
Core Elements:

- **Agent**: The learner or decision maker.
- **Environment**: The external system the agent interacts with.
- **State**: A representation of the current situation.
- **Action**: Choices the agent can make.
- **Reward**: Feedback signal for evaluating actions.

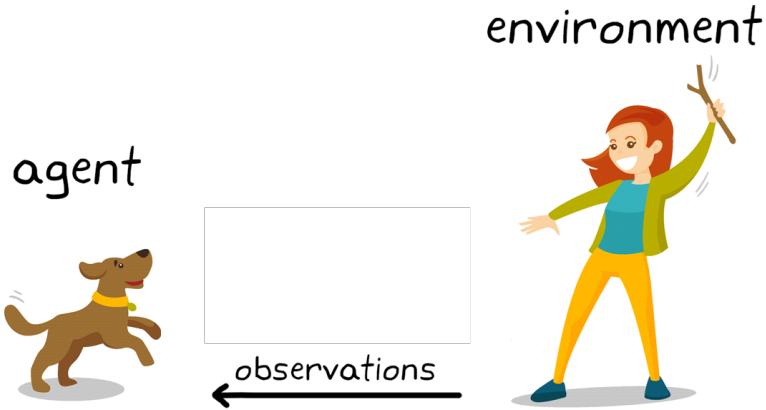
Introduction (cont.)

agent

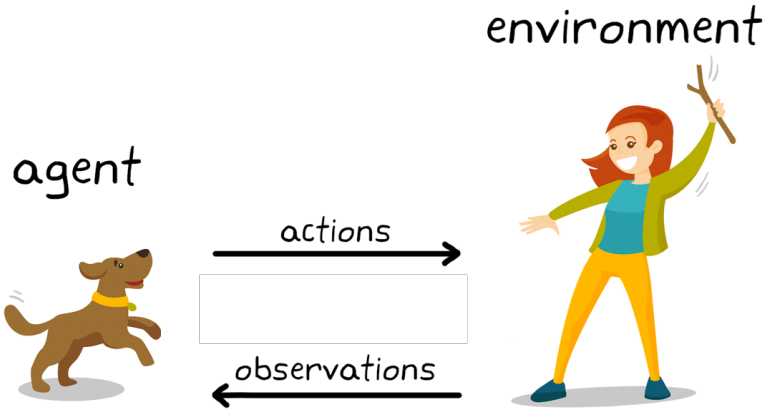
Introduction (cont.)



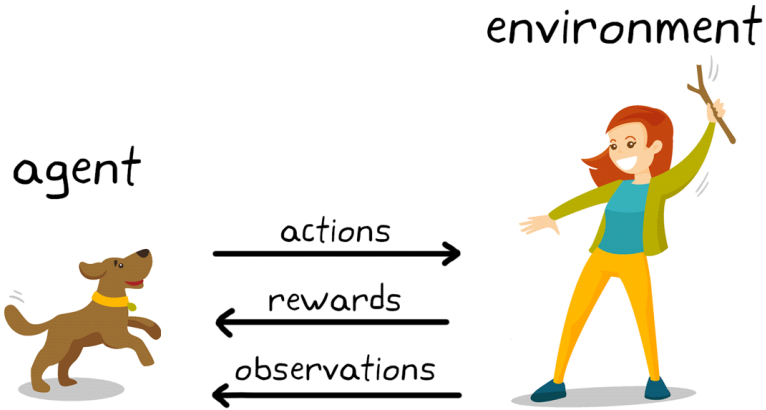
Introduction (cont.)



Introduction (cont.)



Introduction (cont.)

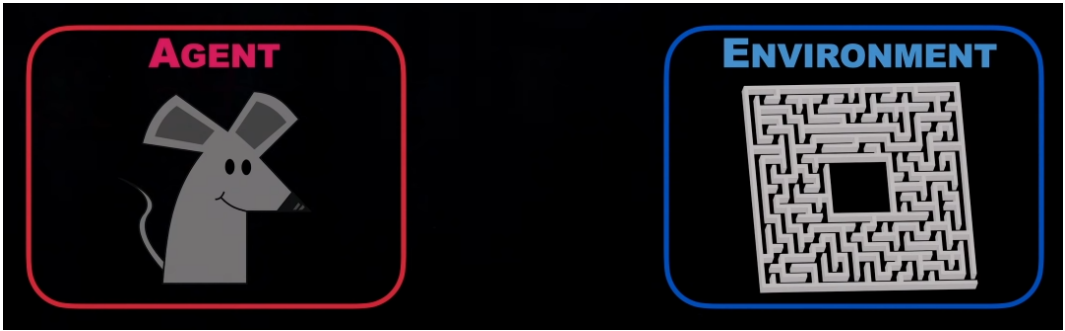


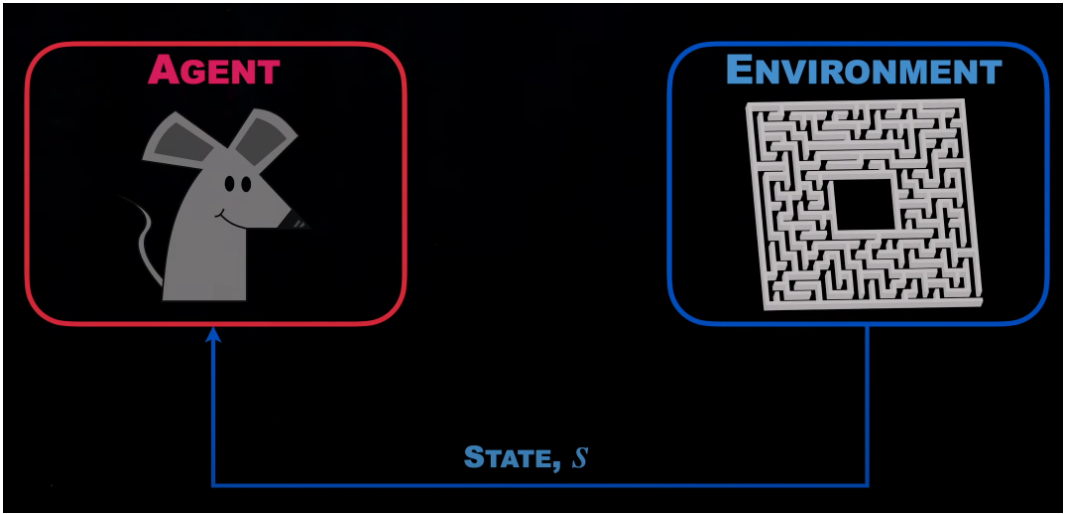
Key Concepts:

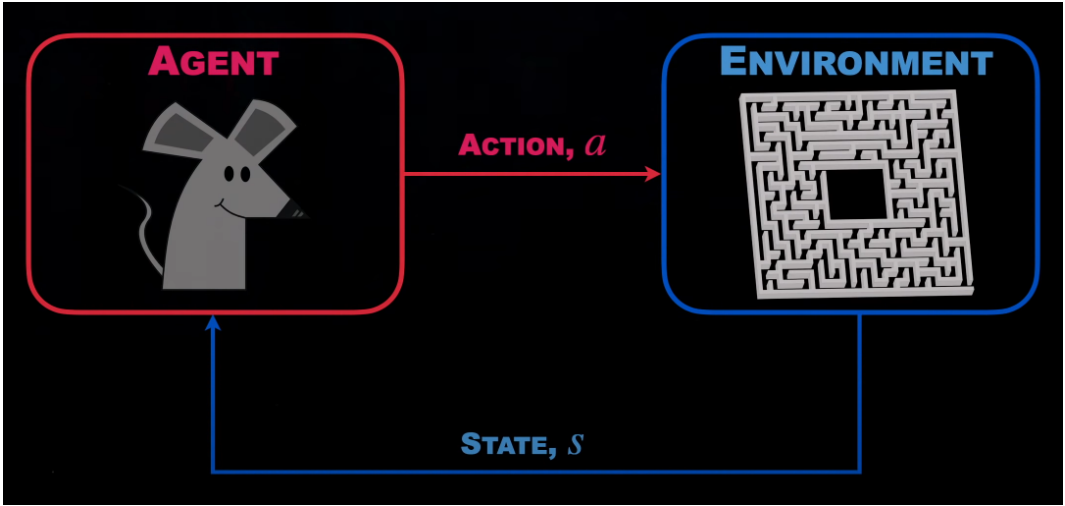
- Agent interacts with environment.
- Learns by trial and error.

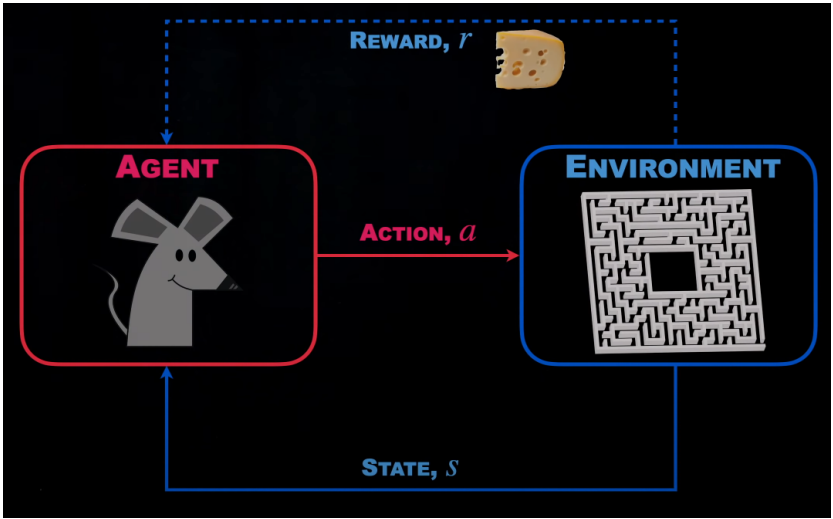
How does it work?

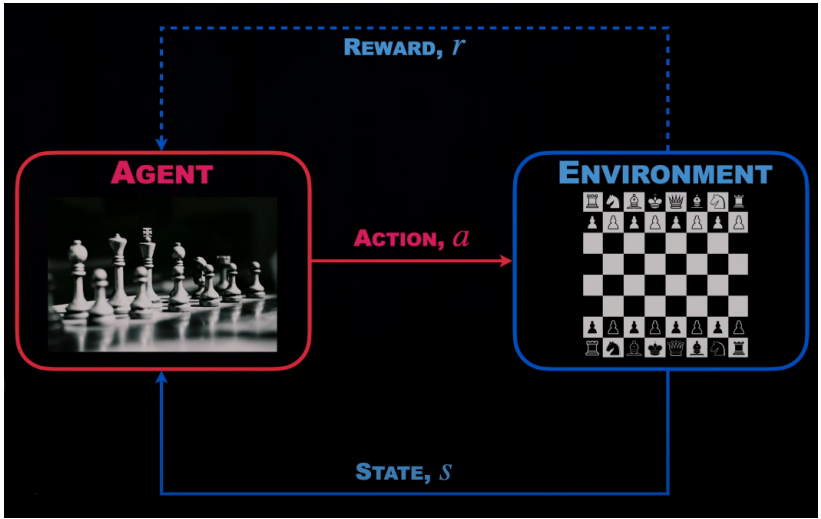
- Agent gets rewards or penalties.
- Goal: Maximize cumulative reward (a.k.a. return).

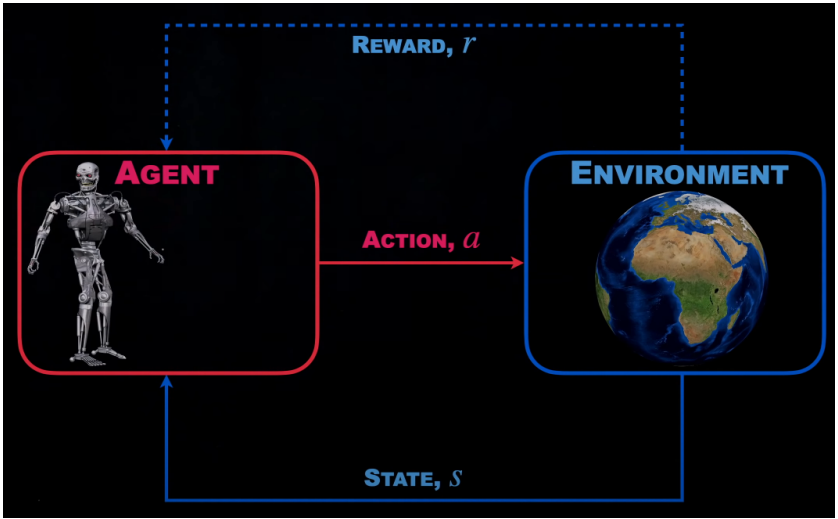

AGENT
ENVIRONMENT

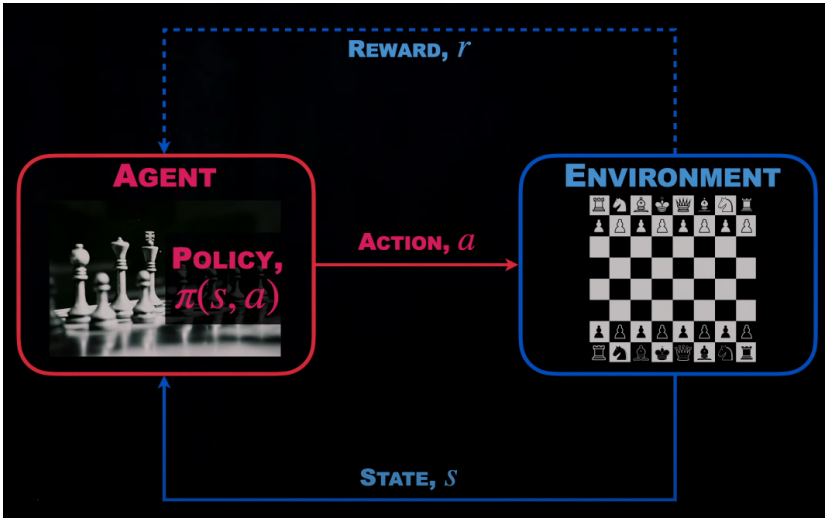

AGENT
ENVIRONMENT
STATE, S

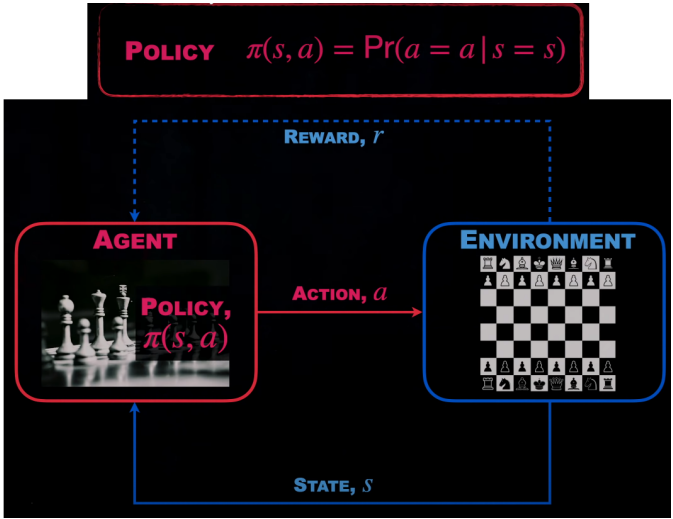

AGENT
ACTION, a
ENVIRONMENT
STATE, s

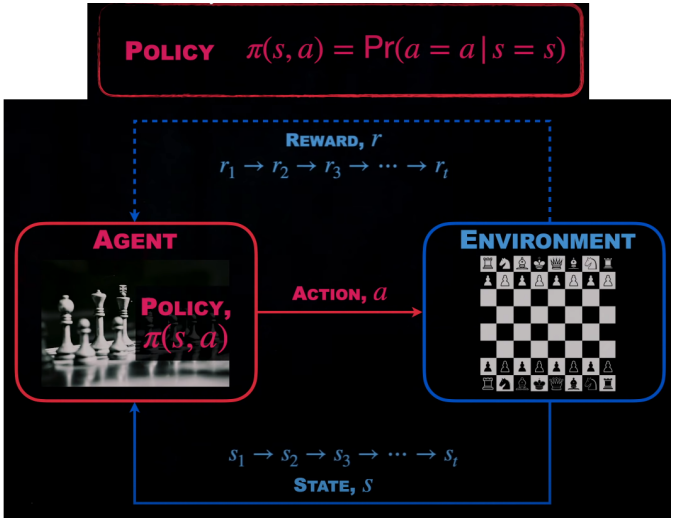

REWARD, r
AGENT
ACTION, a
ENVIRONMENT
STATE, s

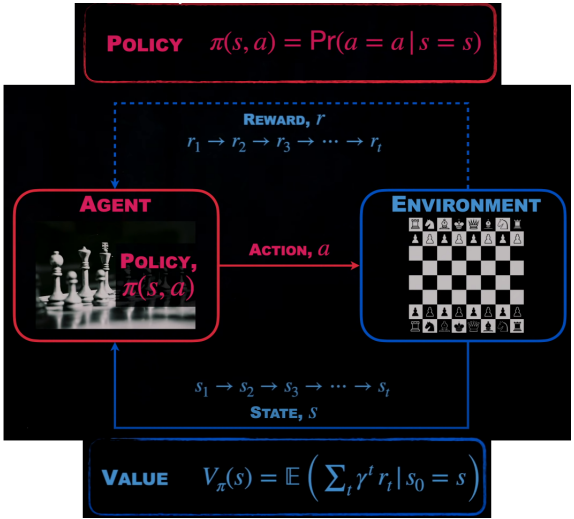

REWARD, r
AGENT
ENVIRONMENT
ACTION, a
STATE, s

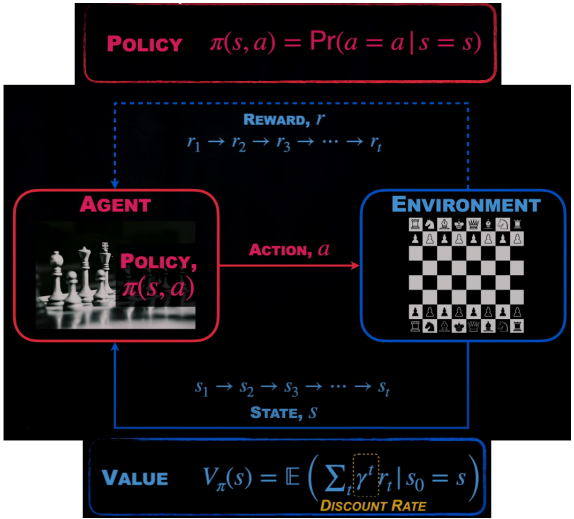

REWARD, r
AGENT
ENVIRONMENT
ACTION, a
STATE, s

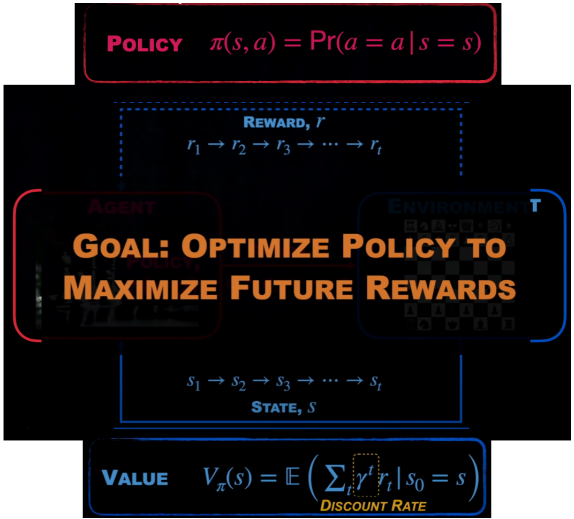

REWARD, r
AGENT
POLICY, $\pi(s, a)$
ACTION, a
ENVIRONMENT
STATE, s


POLICY $\pi(s,a) = \Pr(a = a \mid s = s)$
REWARD, r
AGENT
POLICY, $\pi(s,a)$
ACTION, a
ENVIRONMENT
STATE, s


POLICY $\pi(s,a) = \Pr(a = a \mid s = s)$
REWARD, r
$r_1 \to r_2 \to r_3 \to \cdots \to r_t$
AGENT
POLICY, $\pi(s,a)$
ACTION, a
ENVIRONMENT
$s_1 \to s_2 \to s_3 \to \cdots \to s_t$
STATE, s


POLICY $\pi(s,a) = \Pr(a = a \mid s = s)$
REWARD, r
$r_1 \to r_2 \to r_3 \to \cdots \to r_t$
AGENT
POLICY, $\pi(s,a)$
ACTION, a
ENVIRONMENT
$s_1 \to s_2 \to s_3 \to \cdots \to s_t$
STATE, s
VALUE $V_\pi(s) = \mathbb{E}\left(\sum_t \gamma^t r_t \mid s_0 = s\right)$


POLICY $\pi(s,a) = \Pr(a = a \mid s = s)$
REWARD, r
$r_1 \to r_2 \to r_3 \to \cdots \to r_t$
AGENT
POLICY, $\pi(s,a)$
ACTION, a
ENVIRONMENT
$s_1 \to s_2 \to s_3 \to \cdots \to s_t$
STATE, s
VALUE $V_\pi(s) = \mathbb{E}\left(\sum_t \gamma^t r_t \mid s_0 = s\right)$
DISCOUNT RATE


POLICY $\pi(s,a) = \Pr(a = a \mid s = s)$
REWARD, r
$r_1 \to r_2 \to r_3 \to \cdots \to r_t$
AGENT
POLICY
ENVIRONMENT
GOAL: OPTIMIZE POLICY TO MAXIMIZE FUTURE REWARDS
$s_1 \to s_2 \to s_3 \to \cdots \to s_t$
STATE, s
VALUE $V_\pi(s) = \mathbb{E}\left(\sum_t \gamma^t r_t \mid s_0 = s\right)$
DISCOUNT RATE

Supervised Learning vs. Reinforcement Learning

Supervised Learning

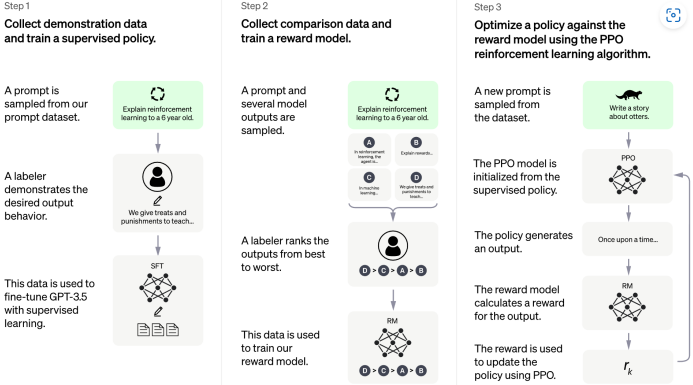
- Given labeled data: $\{(x_i, y_i)\}$, learn $f(x) \approx y$
- Directly told what to output
- Inputs x are independently, identically distributed (i.i.d.)

Reinforcement Learning

- Learn behavior $\pi(a|s)$
- Learn from experience, indirect feedback
- Data not i.i.d.: actions affect future observations

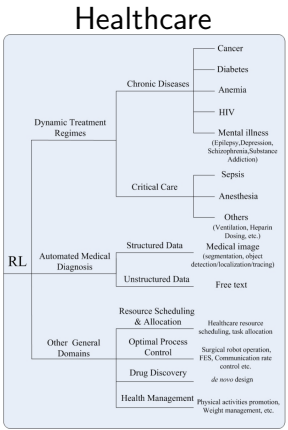
Examples of Reinforcement Learning

ChatGPT



https://openai.com/blog/chatgpt

Examples of Reinforcement Learning (cont.)

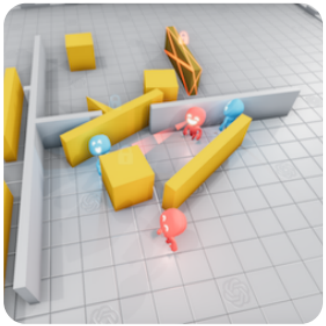


Yu, et al. Reinforcement Learning in Healthcare: A Survey

Examples of Reinforcement Learning (cont.)

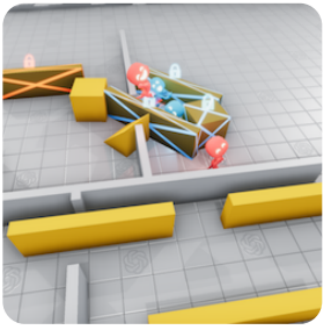
Multiagent Hide & Seek

(a) Running and Chasing



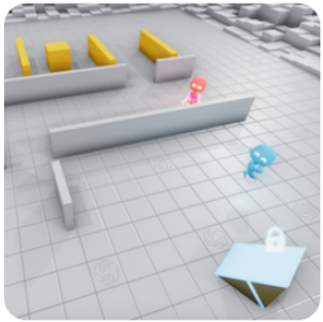
(b) Fort Building

(c) Ramp Use

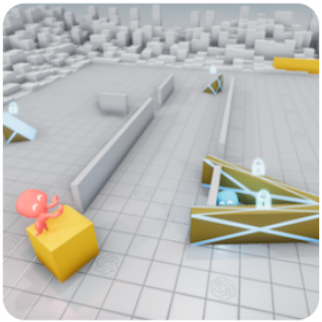


Examples of Reinforcement Learning (cont.)

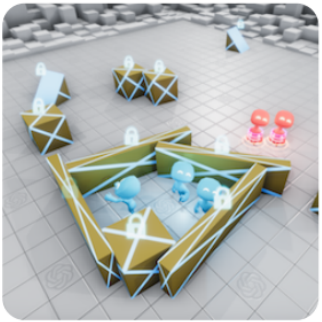
Multiagent Hide & Seek



(d) Ramp Defense



(e) Box Surfing



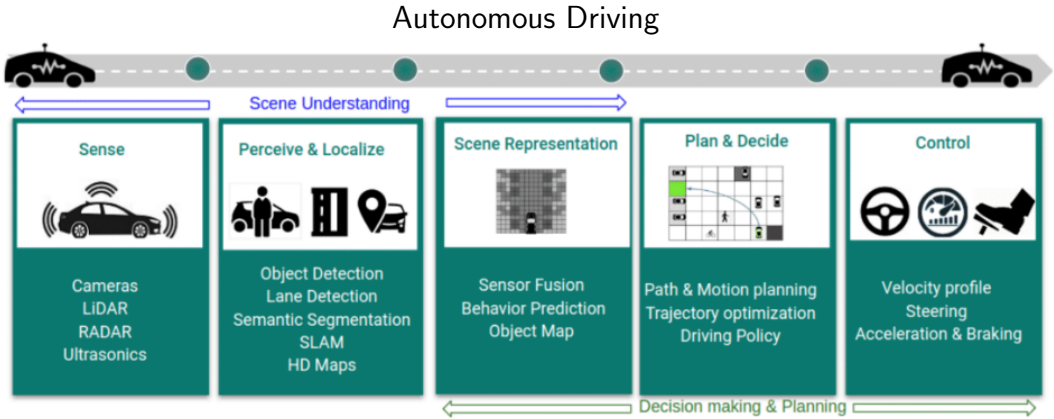
(f) Surf Defense

Examples of Reinforcement Learning (cont.)

Gran Turismo GTSophie

https://www.gran-turismo.com/us/gran-turismo-sophy/technology/

Examples of Reinforcement Learning (cont.)



Kiran, et al. Deep Reinforcement Learning for Autonomous Driving: A Survey

Examples of Reinforcement Learning (cont.)

AlphaGo

'AlphaGo' AI Scores Narrow Win Against Ke Jie, World's Top 'Go' Player

By Lucian Armasu published May 23, 2017

Comments (4)

https://www.tomshardware.com/news/alphago-narrow-win-ke-jie,34486.html

Examples of Reinforcement Learning (cont.)

Muzero

MuZero learns the rules of the game, allowing it to also master environments with unknown dynamics.
(Dec 2020, Nature)

https://www.deepmind.com/blog/muzero-mastering-go-chess-shogi-and-atari-without-rules

Minecraft

AI learns how to play Minecraft by watching videos

Open AI has trained a neural network to play Minecraft by Video PreTraining (VPT) on a massive unlabeled video dataset of human Minecraft play, while using just a small amount of labeled contractor data.

With a bit of fine-tuning, the AI research and deployment company is confident that its model can learn to craft diamond tools, a task that usually takes proficient humans over 20 minutes (24,000 actions). Its model uses the native human interface of keypresses and mouse...

29 June 2022 | Artificial Intelligence

https://www.artificialintelligence-news.com/2022/06/29/ai-learns-how-to-play-minecraft-by-watching-videos/

Examples of Reinforcement Learning (cont.)

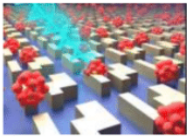
More Examples

Reinforcement learning: From board games to protein design

Scientists have successfully applied reinforcement learning to a challenge in molecular biology. The team of researchers developed powerful new protein design software adapted from a strategy proven adept at board games like ...

BIOTECHNOLOGY

APR 20, 2023 0 37



Chiral detection of biomolecules based on reinforcement learning

As one of the basic physical properties, chirality plays an important role in many fields. Especially in biomedical chemistry, the discrimination of enantiomers is a very important research subject. Most biomolecules exhibit ...

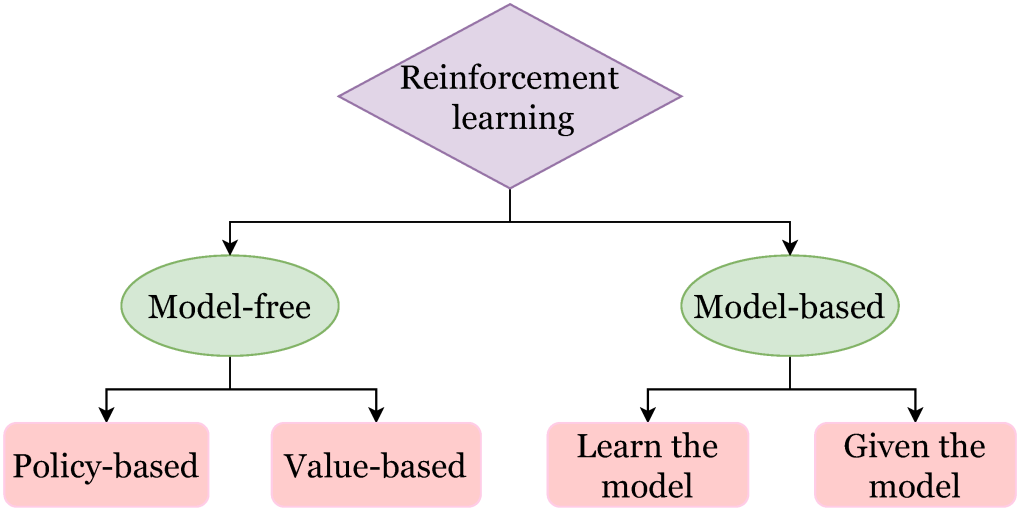
OPTICS & PHOTONICS

FEB 22, 2023 0 4

https://phys.org/tags/reinforcement+learning/

Reinforcement Learning: **Types**

Types of Reinforcement Learning



Types of Reinforcement Learning (cont.)

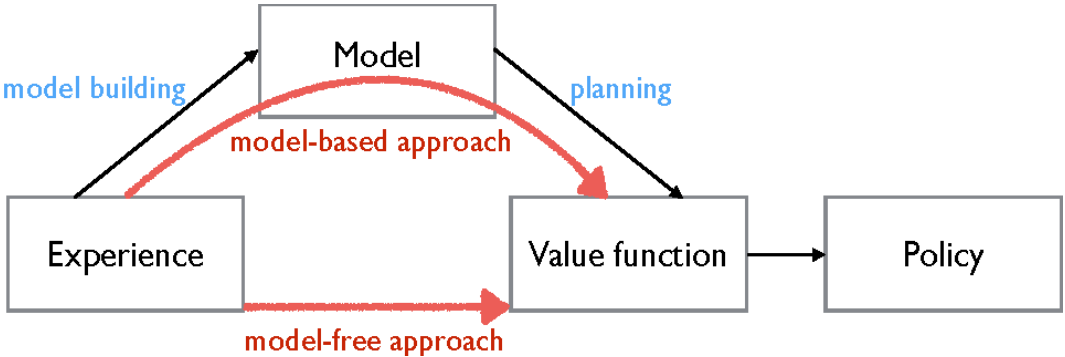
Model-based vs. Model-free

- **Model-based:** The agent builds or uses a model of the environment to plan actions.
- **Model-free:** The agent learns to act without explicitly modeling the environment.

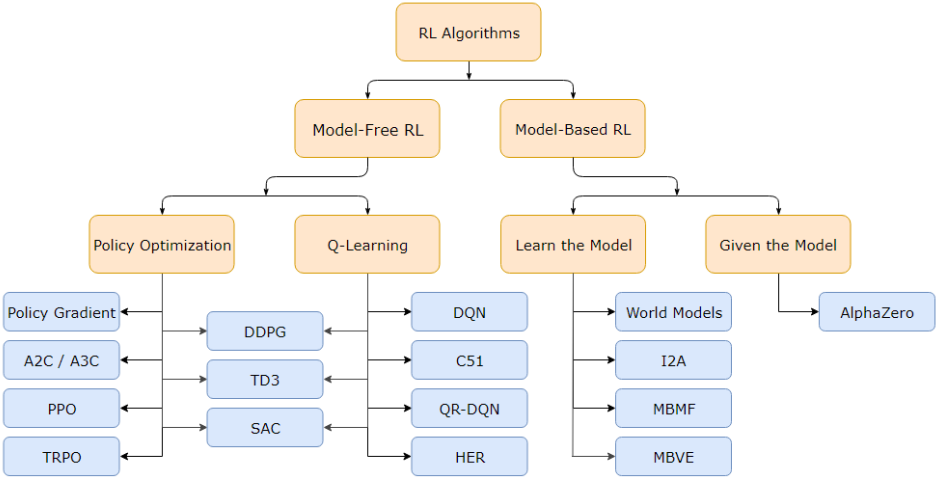
Value-based vs. Policy-based

- **Value-based:** The agent learns a value function (e.g., Q-learning) to evaluate actions or states.
- **Policy-based:** The agent directly learns a policy that maps states to actions (e.g., Policy Gradient methods).

Types of Reinforcement Learning (cont.)



Types of Reinforcement Learning (cont.)

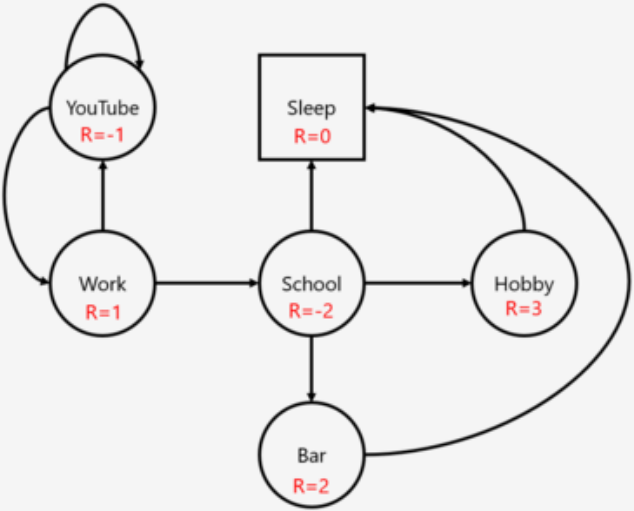


Reinforcement Learning: **Markov Decision Processes (MDPs)**

Mathematical Formulation of the RL Problem

- **Markov property:** The current state completely characterizes the state of the world; that is, the future is independent of the past given the present.
- **Defined by:** $(\mathcal{S}, \mathcal{A}, R, P, \gamma)$
  - $\mathcal{S}$: set of possible states
  - $\mathcal{A}$: set of possible actions
  - $R(s, a)$: reward distribution given a (state, action) pair
  - $P(s' \mid s, a)$: transition probability, i.e., distribution over next state given a (state, action) pair
  - *Notation:* we write $r(s, a) = \mathbb{E}[R(s, a)]$ for the expected reward and $p(s' \mid s, a)$ for the transition density. These lowercase forms appear in the Bellman equations and throughout the rest of the course.
  - γ: discount factor
- Assumes the **Markov property**: the next state depends only on the current state and action.

Mathematical Formulation of the RL Problem (cont.)



Mathematical Formulation of the RL Problem (cont.)

- At time step $t = 0$, the environment samples the initial state $s_0 \sim p(s_0)$.
- Then, for $t = 0$ until termination:
  - The agent selects action a_t.
  - The environment samples reward $r_t \sim R(\cdot|s_t, a_t)$.
  - The environment samples the next state $s_{t+1} \sim p(\cdot|s_t, a_t)$.
  - The agent receives reward r_t and next state s_{t+1}.
- A policy π is a function from $\mathcal{S}$ to $\mathcal{A}$ that specifies which action to take in each state.
- **Objective:** Find a policy $\pi^\star$ that maximizes the cumulative discounted reward: $\sum_{t \geq 0} \gamma^t r_t$.

Mathematical Formulation of the RL Problem (cont.)

- The agent and the environment interact in a time-sequenced loop:
  - The agent observes the current state and reward, then selects an action.
  - The environment responds by producing the next state and a scalar reward.
- Each state satisfies the **Markov property**:
  - The next state and reward depend only on the current state and action.
  - The current state captures all relevant information from the history.
  - The current state is a sufficient statistic for predicting the future (given the action).
- The goal of the agent is to maximize the expected sum of all future rewards by controlling the policy function $\pi : \mathcal{S} \to \mathcal{A}$.
- This setup forms a dynamic, time-sequenced control system under uncertainty.

When the Markov Property Fails

- The Markov property assumes the current state captures everything relevant from history
- Examples where it is violated:
  - A robot that only has a camera (partial view of the world) — the image alone does not tell you velocity or what is behind the robot
  - A card game where you cannot see the opponent's hand
  - A language model generating text — the "state" of recent tokens does not fully capture long-range dependencies
- When Markov fails we are in a **Partially Observable MDP (POMDP)** — the agent receives an observation o_t rather than the true state s_t
- Common workaround: stack recent observations, or use recurrent networks to approximate the hidden state
- This will be relevant on Day 9 when we discuss RLHF for language models

Real-World Problems that Fit the MDP Framework

- Self-driving vehicles: choosing speed and steering to optimize safety and travel time
- Playing chess: receiving a reward only at the end of the game
- Complex logistical operations, such as managing movements in a warehouse
- Enabling a humanoid robot to walk or run on challenging terrains
- Managing an investment portfolio over time
- Controlling a power station efficiently
- Making optimal decisions during a football match
- Developing strategies to win an election (a high-complexity MDP)

Why Are These Problems Hard?

- The state space can be large or complex (involving many variables).
- Sometimes, the action space is also large or complex.
- There is no direct feedback on "correct" actions (the only feedback is the reward).
- Time-sequenced complexity: actions influence future states and actions.
- Actions can have delayed consequences (delayed rewards).
- The agent often does not know the model of the environment.
- The "model" refers to the probabilities of state transitions and rewards.
- Thus, the agent has to learn the model and solve for the optimal policy.
- Agent actions need to trade off between "exploration" and "exploitation".

Why Tabular Methods Break Down

- Q-learning stores a Q-value for every (state, action) pair in a table
- Simple grid world: 12 states $\times$ 4 actions $=$ 48 entries — completely feasible
- Atari game: raw pixel input gives $\sim 10^{70000}$ possible states — storing a table is impossible
- Real robot with continuous joint angles and velocities: infinite state space
- This is the **curse of dimensionality**
- Solution: approximate $Q(s, a)$ with a parameterized function, e.g., a neural network $Q_\theta(s, a)$
- This is exactly what **Deep Q-Networks (DQN)** do — which we cover on Day 2

Reinforcement Learning: **Policy**

Policy

- Policy π determines how the agent chooses actions
- $\pi : \mathcal{S} \rightarrow \mathcal{A}$, mapping from states to actions
- Deterministic Policy:

$$\pi(s) = a$$

- Stochastic Policy:

$$\pi(a|s) = Pr(a_t = a|s_t = s)$$

Parameterized Policies

- In tabular settings, we can store $\pi(a|s)$ for every state explicitly
- In large or continuous state spaces this is infeasible — same curse of dimensionality problem
- Instead, represent the policy as a parameterized function: $\pi_\theta(a|s)$
- θ are learnable parameters (e.g., weights of a neural network)
- The network takes state s as input and outputs a probability distribution over actions
- Learning RL then becomes: find θ^* that maximizes expected return
- Two broad approaches:
  - Learn a value function and derive the policy from it (Days 2, 5, 6)
  - Directly optimize θ to maximize return — Policy Gradient methods (Day 3 onwards)

The Optimal Policy

- We want to find optimal policy $\pi^\star$ that maximizes the sum of reward
- But, how do we handle the randomness (initial state, transition probability...)?

The Optimal Policy

- We want to find optimal policy $\pi^\star$ that maximizes the sum of reward
- But, how do we handle the randomness (initial state, transition probability. . .)?
- **Solution**: Maximize the expected sum of rewards!
- Formally,

$$\pi^\star = \arg\max_\pi \mathbb{E}\left[\sum_{t\geq 0} \gamma^t r_t | \pi\right] \quad \text{with} \quad s_0 \sim p(s_0), a_t \sim \pi(.|s_t), s_{t+1} \sim p(.|s_t, a_t)$$

Reinforcement Learning: **Value Function**

Value Function

- How good is a state?

Value Function

- How good is a state?
- The **value function** at state s, is the expected cumulative reward from following the policy from state s:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t \geq 0} \gamma^{t} r_{t} \mid s_{0} = s, \pi\right]$$

- Computing the value function is generally impractical, but we can try to approximate (learn) it
- The benefit is credit assignment: see directly how an action affects future returns rather than wait for rollouts

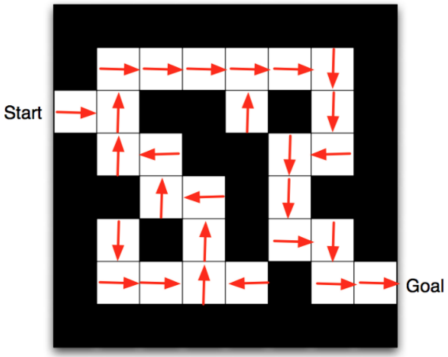
The Discount Factor γ

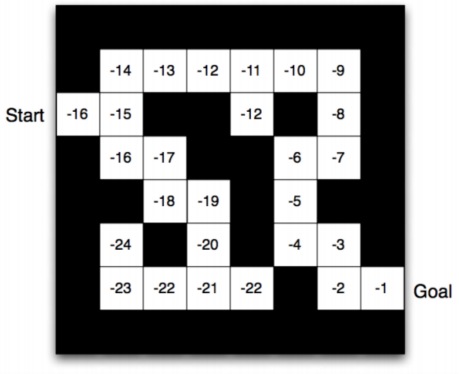
- $\gamma \in [0, 1]$ controls how much the agent values future rewards
- $\gamma = 0$: agent is completely myopic, only cares about immediate reward
- $\gamma \to 1$: agent values future rewards almost as much as immediate ones
- Practical implication: $\gamma = 0.99$ means a reward 100 steps away is worth $\sim$37% of its face value
- Why not $\gamma = 1$? In continuing tasks (no terminal state), the sum of rewards can diverge to infinity, making the objective undefined
- Rule of thumb: use γ closer to 1 for long-horizon tasks, smaller γ for tasks where short-term feedback is reliable

Episodic vs. Continuing Tasks

- **Episodic**: interaction breaks into episodes with a clear terminal state (e.g., a chess game ends in win/loss/draw, a robot falls over). Each episode starts fresh from an initial state distribution
- **Continuing**: interaction goes on indefinitely with no natural endpoint (e.g., a stock trading agent, a power grid controller). Discounting is essential here to keep returns finite
- Most benchmark environments (CartPole, Atari, MuJoCo) are episodic
- This distinction affects how we compute returns and will matter when we derive policy gradient methods on Day 3

Value Function





Q-Value Function

- Can we use a value function to choose actions?

$$\arg\max_a r(s_t, a) + \gamma \mathbb{E}_{p(s_{t+1}|s_t, a_t)} \left[V^\pi(s_{t+1}) \right]$$

Q-Value Function

- Can we use a value function to choose actions?

$$\arg\max_a r(s_t, a) + \gamma \mathbb{E}_{p(s_{t+1}|s_t,a_t)} \left[V^\pi(s_{t+1}) \right]$$

- **Problem:** this requires taking the expectation with respect to the environment's dynamics, which we don't have direct access to!
- Instead, learn a Q-value function.

Q-Value Function

- The Q-value function at state s and action a, is the expected cumulative reward from taking action a in state s and then following the policy:

$$Q^{\pi}(s,a) = \mathbb{E}\left[\sum_{t\geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi\right]$$

- Relationship:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) Q^{\pi}(s,a)$$

- Optimal Action:

$$\arg\max_{a} Q^{\pi}(s,a)$$

The Advantage Function

- Recall: $V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a)$
- The advantage function measures how much better action a is compared to the average action in state s under policy π:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- Intuition: if $A^\pi(s,a) > 0$, action a is better than what the policy would do on average — we should do it more
- If $A^\pi(s,a) < 0$, action a is worse than average — we should do it less
- $A^\pi(s,a) = 0$ for all a when the policy is already optimal in that state
- This function will be central to Actor-Critic methods on Day 4 and PPO

Bellman Equation

- The Bellman Equation is a recursive formula for the Q-value function:

$$Q^{\pi}(s,a) = r(s,a) + \gamma \mathbb{E}_{p(s'|s,a)\pi(a'|s')}\left[Q^{\pi}(s',a')\right]$$

- There are various Bellman equations, and most RL algorithms are based on repeatedly applying one of them.

  The key insight is that the value of a state can be expressed in terms of the values of the states you might end up in next. This recursion is what makes the Bellman equation powerful — and computable. You do not need to look infinitely far into the future; you only need to look one step ahead and trust your current value estimate for the rest.

Bellman Equations in Detail

- For the value function $V^\pi(s)$:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) \left[r(s,a) + \gamma V^\pi(s') \right]$$

- For the Q-value function $Q^\pi(s,a)$:

$$Q^\pi(s,a) = \sum_{s'} p(s'|s,a) \left[r(s,a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right]$$

- Most RL algorithms are based on repeatedly applying one of these equations.

Optimal Bellman Equation

- The optimal policy $\pi^\star$ is the one that maximizes the expected discounted reward, and the optimal Q-value function $Q^\star$ is the Q-value function for $\pi^\star$
- The Optimal Bellman Equation gives a recursive formula for $Q^\star$:

$$Q^\star(s,a) = r(s,a) + \gamma \mathbb{E}_{p(s'|s,a)} \left[\max_{a'} Q^\star(s',a') \right]$$

- This system of equations characterizes the optimal Q-value function. So maybe we can approximate $Q^\star$ by trying to solve the optimal Bellman equation!
- **Intuition**: If the optimal state-action values for the next time-step $Q^\star(s',a')$ are known, then the optimal strategy is to take the action that maximizes the expected value of $r(s,a) + \gamma Q^\star(s',a')$

Reinforcement Learning: **Q-Learning**

Q-Learning

- Q-learning is a **model-free** reinforcement learning algorithm
- It **learns the value of actions in states**, known as the **Q-value function**
- The goal is to find the optimal policy that maximizes the expected cumulative reward
- It can be used in environments with unknown dynamics, making it suitable for a wide range of problems

Q-Learning (cont.)

- Let Q be an action-value function which hopefully approximates $Q^\star$
- Q-learning is an algorithm that repeatedly adjusts Q to minimize the Bellman error
- Each time we sample consecutive states and actions (s_t, a_t, s_{t+1})

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]}_{\text{Bellman Error}}$$

From Bellman Equation to Q-Learning: The Bridge (1/2)

- The optimal Bellman equation tells us what $Q^\star$ must satisfy:

$$Q^\star(s,a) = r(s,a) + \gamma \max_{a'} Q^\star(s',a')$$

- We do not know $Q^\star$ — but we can define a target for any current estimate Q:

$$\text{Target} = r(s,a) + \gamma \max_{a'} Q(s',a')$$

- If $Q = Q^\star$ then Target $= Q(s,a)$ — there is no error
- If $Q \neq Q^\star$ then (Target $- Q(s,a)$) is the **Bellman error** — the gap between what our estimate predicts and what the Bellman equation says it should be

From Bellman Equation to Q-Learning: The Bridge (2/2)

- Q-learning works by repeatedly sampling transitions and nudging Q toward the target using gradient descent (or a simple update rule)
- The learning rate α controls the size of each nudge
- This same principle — define a Bellman target, minimize the error — underlies DQN, Double DQN, and most value-based methods in Days 2 and 5

Exploration-Exploitation Tradeoff

- Why does a purely greedy policy fail? If the agent always exploits its current best estimate of Q, it will never visit state-action pairs it has not tried yet. Those unvisited pairs might have higher Q-values, but the agent will never discover this. In the worst case, the agent gets stuck in a local optimum — a decent but suboptimal strategy it never escapes.
- Notice: Q-learning only learns about the states and actions it visits.
- Exploration-exploitation tradeoff: the agent should sometimes pick suboptimal actions in order to visit new states and actions.

ε-Greedy Policy

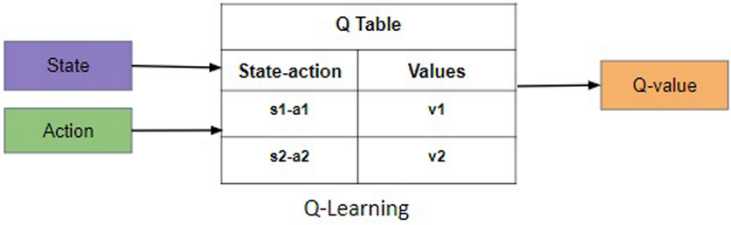
- Simple solution: ε-greedy policy
  - With probability $1 - \varepsilon$, choose the optimal action according to Q
  - With probability ε, choose a random action
- ε-greedy policy still widely used today regardless of simplicity
- Key practical consideration: ε is often annealed over training — starting high (lots of exploration early on) and decaying toward a small value (mostly exploitation once Q is well-estimated). We will revisit exploration more rigorously on Day 7.

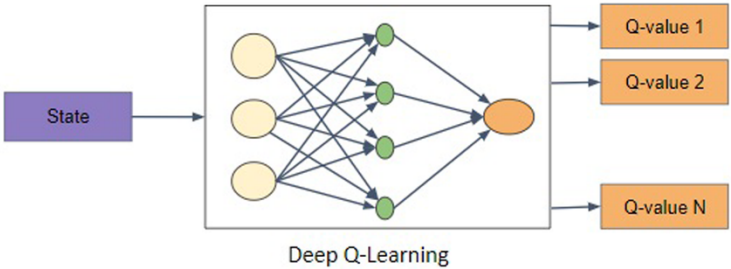
Q-Learning Algorithm

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
    $S \leftarrow S'$;
  until S is terminal

Convergence Properties

- Q-learning converges to the optimal action-value function $Q^\star$ under certain conditions:
  - **Sufficient exploration:** Every state-action pair is visited infinitely often.
  - **Learning rate:** The learning rate α decays appropriately, i.e., $\alpha \to 0$ as the number of visits increases, but $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$.
  - **Finite MDP:** The environment is a finite Markov Decision Process.
- In plain terms: α must be large enough that the agent keeps learning ($\sum \alpha_t = \infty$, so it does not stop updating), but must shrink fast enough that the random noise in updates averages out ($\sum \alpha_t^2 < \infty$). In practice, a constant small α (e.g., 0.001) is almost always used instead — convergence guarantees are lost but the algorithm works well empirically.
- Under these conditions, Q-learning is guaranteed to converge to $Q^\star$ with probability 1.


Q Table
State
Action
State-action
Values
s1-a1
v1
s2-a2
v2
Q-value
Q-Learning


State
Q-value 1
Q-value 2
Q-value N
Deep Q-Learning

Reinforcement Learning: **Limitations**

Limitations of Reinforcement Learning

- **Sample inefficiency**: Requires a large number of interactions with the environment to learn effectively.
- **Stability issues with function approximation**: Training with neural networks can be unstable and sensitive to hyperparameters.
- **Sparse or delayed rewards**: Learning is difficult when rewards are infrequent or delayed.
- **Requires careful reward design**: Poorly designed rewards can lead to unintended behaviors.
- **Poor generalization across tasks**: Policies often do not transfer well to new or slightly different tasks.

Reinforcement Learning: **Summary**

Summary

- Reinforcement Learning (RL) is a powerful paradigm for **sequential decision-making**.
- Markov Decision Processes (MDPs) provide a **mathematical foundation**.
- Q-learning enables **model-free control**.
- Despite its power, RL has **significant limitations**.
- Research is ongoing to make RL **scalable, robust, and safe**.

Course Road Map: Where Today Fits

Day	Topic	Builds on Today's...
2	DQN	Q-learning + parameterized Q_θ + Bellman error
3	Policy Gradient	Parameterized policy π_θ + return objective
4	Actor-Critic / PPO	Advantage function + both V and π
5	DDPG	Q-learning extended to continuous actions
6	SAC	Bellman equation + entropy term added to reward
7	Exploration	ε-greedy extended + bandit formalisation
8	Model-Based	MDP transition model $P(s'\|s,a)$ made explicit
9	RLHF	Policy π_θ + reward design + partial observability
10	Frontiers	All of the above

Reinforcement Learning: **References**

References

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.).

[2] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*.

[3] Silver, D., Huang, A., Maddison, C. J., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*.

[4] Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline Reinforcement Learning: Tutorial, Review, and Perspectives. *arXiv:2005.01643*.

[5] David Silver's RL Course: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html

[6] Berkeley CS285: Deep RL https://rail.eecs.berkeley.edu/deeprlcourse/

References (cont.)

[7] Emma Brunskill, Stanford CS234: Reinforcement Learning
https://web.stanford.edu/class/cs234/